

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - ANSWER KEY

Course Name: Operating Systems

Course Code: CSA0403

Course Outcome (CO3): Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements.

Question 1

Simulate translation of a logical address into a physical address using base and limit registers

Theory / Explanation

In the base–limit register method, the base register stores the starting physical address of a process, and the limit register stores the size of the process's logical address space. A logical address is valid only if it is less than the limit; the physical address is then obtained by adding the base value to the logical address. If the logical address is greater than or equal to the limit, a trap (addressing error) is generated and access is denied, thereby protecting other processes in memory.

Pseudocode

```
BEGIN
  READ base register value (BASE)
  READ limit register value (LIMIT)
  READ logical address (LA)
  IF LA >= 0 AND LA < LIMIT THEN
    PA = BASE + LA
    PRINT "Valid address, Physical Address = " PA
  ELSE
    PRINT "Invalid address - Trap to Operating System"
  END IF
END
```

C Program (Code)

```
#include <stdio.h>

int main() {
    int base, limit, logical, physical;

    printf("Enter Base Register value: ");
    scanf("%d", &base);
    printf("Enter Limit Register value: ");
    scanf("%d", &limit);
    printf("Enter Logical Address: ");
    scanf("%d", &logical);

    if (logical >= 0 && logical < limit) {
        physical = base + logical;
        printf("\nValid Address.\n");
        printf("Physical Address = %d\n", physical);
    } else {
        printf("\nInvalid Address. Trap: Addressing Error!\n");
    }
    return 0;
}
```

Sample Input

```
Enter Base Register value: 5000
Enter Limit Register value: 1000
Enter Logical Address: 300
```

Sample Output

Valid Address.

Physical Address = 5300

(For Logical Address = 1200)

Invalid Address. Trap: Addressing Error!

Question 2

Demonstrate the paging memory management scheme

Theory / Explanation

Paging divides logical memory into fixed-size blocks called pages and physical memory into blocks of the same size called frames. A page table maps each page number to a frame number. The logical address is split into a page number and an offset; the page number indexes the page table to get the frame number, and the physical address is computed as $(\text{frame_number} * \text{page_size}) + \text{offset}$.

Pseudocode

```
BEGIN
  READ page_size
  READ number_of_pages
  FOR i = 0 TO number_of_pages-1
    READ page_table[i] // frame number for page i
  END FOR
  READ logical_address
  page_number = logical_address / page_size
  offset = logical_address MOD page_size
  IF page_number < number_of_pages THEN
    frame = page_table[page_number]
    physical_address = frame * page_size + offset
    PRINT physical_address
  ELSE
    PRINT "Invalid page number"
  END IF
END
```

C Program (Code)

```
#include <stdio.h>

int main() {
  int page_size, n, page_table[20];
  int logical, page_number, offset, physical;

  printf("Enter page size: ");
  scanf("%d", &page_size);
  printf("Enter number of pages: ");
  scanf("%d", &n);

  for (int i = 0; i < n; i++) {
    printf("Enter frame number for page %d: ", i);
    scanf("%d", &page_table[i]);
  }

  printf("Enter logical address: ");
  scanf("%d", &logical);

  page_number = logical / page_size;
  offset = logical % page_size;

  if (page_number < n) {
    physical = page_table[page_number] * page_size + offset;
    printf("\nPage Number = %d, Offset = %d\n", page_number, offset);
    printf("Physical Address = %d\n", physical);
  }
```

```
    } else {  
        printf("\nError: Page number out of range\n");  
    }  
    return 0;  
}
```

Sample Input

Page size: 512
Number of pages: 4
Page table: [2, 6, 1, 5]
Logical address: 1200

Sample Output

Page Number = 2, Offset = 176
Physical Address = 688

Question 3

Simulate a page table that maps logical addresses to physical addresses

Theory / Explanation

A page table stores the mapping between each page number and its corresponding frame number in physical memory. When a process generates a logical address, the operating system splits it into a page number and offset, looks up the frame number in the page table, and forms the physical address by combining the frame number with the offset, enabling translation of multiple logical addresses at run time.

Pseudocode

```
BEGIN
  READ page_size
  READ number_of_entries
  FOR i = 0 TO number_of_entries-1
    READ page_number, frame_number
    page_table[page_number] = frame_number
  END FOR
  READ number_of_logical_addresses
  FOR each logical_address
    page = logical_address / page_size
    offset = logical_address MOD page_size
    IF page_table[page] exists THEN
      physical = page_table[page] * page_size + offset
      PRINT physical
    ELSE
      PRINT "Page fault / invalid page"
    END IF
  END FOR
END
```

C Program (Code)

```
#include <stdio.h>

#define MAX 50

int main() {
  int page_size, entries, page_table[MAX];
  int page_no, frame_no, addresses, la, page, offset;

  for (int i = 0; i < MAX; i++) page_table[i] = -1;

  printf("Enter page size: ");
  scanf("%d", &page_size);
  printf("Enter number of page table entries: ");
  scanf("%d", &entries);

  for (int i = 0; i < entries; i++) {
    printf("Enter page number and frame number: ");
    scanf("%d %d", &page_no, &frame_no);
    page_table[page_no] = frame_no;
  }

  printf("Enter number of logical addresses to translate: ");
  scanf("%d", &addresses);
```

```
for (int i = 0; i < addresses; i++) {  
    printf("Enter logical address: ");  
    scanf("%d", &la);  
    page = la / page_size;  
    offset = la % page_size;  
    if (page_table[page] != -1) {  
        printf("Page %d -> Frame %d | Physical Address = %d\n",  
            page, page_table[page], page_table[page] * page_size + offset);  
    } else {  
        printf("Page %d not mapped - Invalid reference\n", page);  
    }  
}  
return 0;  
}
```

Sample Input

Page size: 1024
Entries: 3 -> (0,5) (1,3) (2,7)
Logical addresses to translate: 2 -> 2500, 900

Sample Output

Page 2 -> Frame 7 | Physical Address = 7644
Page 0 -> Frame 5 | Physical Address = 6020

Question 4

Simulate the working of a Translation Lookaside Buffer (TLB) and calculate effective access time

Theory / Explanation

A TLB is a small, fast associative cache that stores recent page-to-frame translations. On a TLB hit, the frame number is obtained directly, saving a page-table lookup; on a TLB miss, the page table in memory must be consulted (costing an extra memory access) and the entry is then loaded into the TLB. The Effective Memory Access Time (EMAT) is calculated using the hit ratio: $EMAT = \text{hit_ratio} * (\text{TLB time} + \text{memory time}) + (1 - \text{hit_ratio}) * (\text{TLB time} + 2 * \text{memory time})$.

Pseudocode

```
BEGIN
  READ tlb_time, memory_time
  READ page_reference_string
  TLB = empty list (capacity = TLB_SIZE)
  hits = 0, misses = 0
  FOR each page in reference string
    IF page IN TLB THEN
      hits = hits + 1
    ELSE
      misses = misses + 1
      insert page into TLB (evict oldest if full)
    END IF
  END FOR
  hit_ratio = hits / total_references
  EMAT_with_TLB = hit_ratio*(tlb_time+memory_time) +
                  (1-hit_ratio)*(tlb_time+2*memory_time)
  EMAT_without_TLB = 2 * memory_time
  PRINT hits, misses, EMAT_with_TLB, EMAT_without_TLB
END
```

C Program (Code)

```
#include <stdio.h>

#define TLB_SIZE 4

int tlb[TLB_SIZE], count = 0, front = 0;

int searchTLB(int page) {
  for (int i = 0; i < count; i++)
    if (tlb[i] == page) return 1;
  return 0;
}

void insertTLB(int page) {
  if (count < TLB_SIZE) {
    tlb[count++] = page;
  } else {
    tlb[front] = page;
    front = (front + 1) % TLB_SIZE;
  }
}

int main() {
  int n, pages[50], hits = 0, misses = 0;
```



```

float tlb_time, mem_time;

printf("Enter TLB access time and Memory access time: ");
scanf("%f %f", &tlb_time, &mem_time);
printf("Enter number of page references: ");
scanf("%d", &n);
printf("Enter page reference string: ");
for (int i = 0; i < n; i++) scanf("%d", &pages[i]);

for (int i = 0; i < n; i++) {
    if (searchTLB(pages[i])) {
        hits++;
        printf("Page %d -> TLB HIT\n", pages[i]);
    } else {
        misses++;
        insertTLB(pages[i]);
        printf("Page %d -> TLB MISS\n", pages[i]);
    }
}

float hit_ratio = (float) hits / n;
float emat_with = hit_ratio * (tlb_time + mem_time) +
    (1 - hit_ratio) * (tlb_time + 2 * mem_time);
float emat_without = 2 * mem_time;

printf("\nHits = %d, Misses = %d, Hit Ratio = %.2f\n", hits, misses, hit_ratio);
printf("EMAT with TLB      = %.2f ns\n", emat_with);
printf("EMAT without TLB = %.2f ns\n", emat_without);
return 0;
}

```

Sample Input

```

TLB access time = 5, Memory access time = 100
Number of references = 8
Reference string = 1 2 3 4 1 2 5 1

```

Sample Output

```

Page 1 -> TLB MISS
Page 2 -> TLB MISS
Page 3 -> TLB MISS
Page 4 -> TLB MISS
Page 1 -> TLB HIT
Page 2 -> TLB HIT
Page 5 -> TLB MISS
Page 1 -> TLB HIT

Hits = 3, Misses = 5, Hit Ratio = 0.38
EMAT with TLB      = 130.25 ns
EMAT without TLB = 200.00 ns

```

Question 5

Illustrate memory protection using base and limit registers

Theory / Explanation

Memory protection ensures that a process can access only the memory allocated to it. The base register holds the smallest legal physical address, and the limit register holds the size of the range. Every logical address generated by the CPU is checked against the limit register before being combined with the base register; if the address is outside the valid range, the hardware raises an addressing-error trap and the CPU generates a fault to the operating system, preventing the process from accessing another process's memory.

Pseudocode

```
BEGIN
  READ base, limit
  READ number_of_addresses
  FOR each logical_address
    IF 0 <= logical_address < limit THEN
      physical = base + logical_address
      PRINT "Access allowed, Physical Address = " physical
    ELSE
      PRINT "Access denied - Memory protection violation"
    END IF
  END FOR
END
```

C Program (Code)

```
#include <stdio.h>

int main() {
    int base, limit, n, addr;

    printf("Enter base and limit register values: ");
    scanf("%d %d", &base, &limit);
    printf("Enter number of addresses to check: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter logical address: ");
        scanf("%d", &addr);
        if (addr >= 0 && addr < limit) {
            printf("Access ALLOWED. Physical Address = %d\n", base + addr);
        } else {
            printf("Access DENIED. Memory Protection Violation!\n");
        }
    }
    return 0;
}
```

Sample Input

```
Base = 3000, Limit = 800
Number of addresses = 3
Addresses: 500, 900, 50
```

Sample Output

Access ALLOWED. Physical Address = 3500
Access DENIED. Memory Protection Violation!
Access ALLOWED. Physical Address = 3050

Question 6

Simulate dynamic storage allocation using First Fit, Best Fit and Worst Fit

Theory / Explanation

First Fit allocates the first free block whose size is greater than or equal to the request. Best Fit scans all free blocks and allocates the smallest block that satisfies the request, minimizing leftover space. Worst Fit allocates the largest available block, leaving the biggest possible leftover for future use. Each strategy affects the amount of fragmentation left after allocation.

Pseudocode

```
BEGIN
  READ block_sizes[], process_sizes[]
  // First Fit
  FOR each process p
    FOR each block b (in order)
      IF block[b] >= p AND not allocated THEN
        allocate p to b; mark allocated; BREAK
  // Best Fit
  FOR each process p
    find block with MIN size among blocks >= p, not allocated
    allocate p to that block
  // Worst Fit
  FOR each process p
    find block with MAX size among blocks >= p, not allocated
    allocate p to that block
  PRINT allocation table and remaining memory for each strategy
END
```

C Program (Code)

```
#include <stdio.h>
#include <string.h>

void firstFit(int bsize[], int m, int psize[], int n) {
    int allocation[n], used[m];
    memset(allocation, -1, sizeof(allocation));
    memset(used, 0, sizeof(used));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (!used[j] && bsize[j] >= psize[i]) {
                allocation[i] = j; used[j] = 1; break;
            }
    printf("\n-- First Fit --\n");
    for (int i = 0; i < n; i++)
        if (allocation[i] != -1)
            printf("Process %d (%d) -> Block %d (%d), Remaining = %d\n",
                i+1, psize[i], allocation[i]+1, bsize[allocation[i]],
                bsize[allocation[i]] - psize[i]);
        else
            printf("Process %d (%d) -> Not Allocated\n", i+1, psize[i]);
}

void bestFit(int bsize[], int m, int psize[], int n) {
    int allocation[n], used[m];
    memset(allocation, -1, sizeof(allocation));
    memset(used, 0, sizeof(used));
    for (int i = 0; i < n; i++) {
```

```

        int best = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && bsize[j] >= psize[i])
                if (best == -1 || bsize[j] < bsize[best]) best = j;
            if (best != -1) { allocation[i] = best; used[best] = 1; }
    }
    printf("\n-- Best Fit --\n");
    for (int i = 0; i < n; i++)
        if (allocation[i] != -1)
            printf("Process %d (%d) -> Block %d (%d), Remaining = %d\n",
                i+1, psize[i], allocation[i]+1, bsize[allocation[i]],
                bsize[allocation[i]] - psize[i]);
        else
            printf("Process %d (%d) -> Not Allocated\n", i+1, psize[i]);
}

void worstFit(int bsize[], int m, int psize[], int n) {
    int allocation[n], used[m];
    memset(allocation, -1, sizeof(allocation));
    memset(used, 0, sizeof(used));
    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && bsize[j] >= psize[i])
                if (worst == -1 || bsize[j] > bsize[worst]) worst = j;
            if (worst != -1) { allocation[i] = worst; used[worst] = 1; }
    }
    printf("\n-- Worst Fit --\n");
    for (int i = 0; i < n; i++)
        if (allocation[i] != -1)
            printf("Process %d (%d) -> Block %d (%d), Remaining = %d\n",
                i+1, psize[i], allocation[i]+1, bsize[allocation[i]],
                bsize[allocation[i]] - psize[i]);
        else
            printf("Process %d (%d) -> Not Allocated\n", i+1, psize[i]);
}

int main() {
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int bsize[m];
    printf("Enter block sizes: ");
    for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int psize[n];
    printf("Enter process sizes: ");
    for (int i = 0; i < n; i++) scanf("%d", &psize[i]);

    firstFit(bsize, m, psize, n);
    bestFit(bsize, m, psize, n);
    worstFit(bsize, m, psize, n);
    return 0;
}

```

Sample Input

```

Blocks = [100, 500, 200, 300, 600]
Processes = [212, 417, 112, 426]

```

Sample Output

```
-- First Fit --
Process 1 (212) -> Block 2 (500), Remaining = 288
Process 2 (417) -> Block 5 (600), Remaining = 183
Process 3 (112) -> Block 4 (300), Remaining = 188
Process 4 (426) -> Not Allocated

-- Best Fit --
Process 1 (212) -> Block 4 (300), Remaining = 88
Process 2 (417) -> Block 5 (600), Remaining = 183
Process 3 (112) -> Block 3 (200), Remaining = 88
Process 4 (426) -> Not Allocated

-- Worst Fit --
Process 1 (212) -> Block 5 (600), Remaining = 388
Process 2 (417) -> Not Allocated
Process 3 (112) -> Block 2 (500), Remaining = 388
Process 4 (426) -> Not Allocated
```

Question 7

Simulate swapping between main memory and secondary storage

Theory / Explanation

Swapping temporarily moves an inactive process from main memory to secondary storage (swap space) to free up memory for other processes, and later brings it back (swaps it in) when it needs to run again. The operating system maintains available memory and moves processes in and out based on memory availability and process requests, allowing more processes to be handled than would otherwise fit in physical memory.

Pseudocode

```
BEGIN
  READ total_memory
  available = total_memory
  FOR each event (arrival or removal request)
    IF event = arrival THEN
      IF process_size <= available THEN
        swap-in process; available -= process_size
        PRINT "Process swapped IN"
      ELSE
        select a process to swap-out (e.g., least recently used)
        swap-out that process; available += its size
        swap-in new process; available -= its size
        PRINT "Swapped OUT old, Swapped IN new"
      ELSE IF event = removal THEN
        swap-out process; available += process_size
        PRINT "Process swapped OUT"
      END IF
    END FOR
  END
```

C Program (Code)

```
#include <stdio.h>
#include <string.h>

struct Process { char name[10]; int size; int inMemory; };

int main() {
  int total, n;
  printf("Enter total main memory size: ");
  scanf("%d", &total);
  int available = total;

  printf("Enter number of processes: ");
  scanf("%d", &n);
  struct Process p[n];
  for (int i = 0; i < n; i++) {
    printf("Enter name and size of process %d: ", i+1);
    scanf("%s %d", p[i].name, &p[i].size);
    p[i].inMemory = 0;
  }

  int req;
  printf("Enter number of requests: ");
  scanf("%d", &req);

  for (int r = 0; r < req; r++) {
```

```

char name[10]; char action[10];
printf("Enter process name and action (IN/OUT): ");
scanf("%s %s", name, action);

for (int i = 0; i < n; i++) {
    if (strcmp(p[i].name, name) == 0) {
        if (strcmp(action, "IN") == 0) {
            if (!p[i].inMemory && p[i].size <= available) {
                p[i].inMemory = 1;
                available -= p[i].size;
                printf("%s SWAPPED IN. Memory available = %d\n", name, available);
            } else if (p[i].inMemory) {
                printf("%s already in memory\n", name);
            } else {
                printf("Not enough memory to swap in %s\n", name);
            }
        } else {
            if (p[i].inMemory) {
                p[i].inMemory = 0;
                available += p[i].size;
                printf("%s SWAPPED OUT. Memory available = %d\n", name, available);
            } else {
                printf("%s is not in memory\n", name);
            }
        }
    }
}
return 0;
}

```

Sample Input

Total memory = 1000
 Processes: P1 400, P2 300, P3 500
 Requests: P1 IN, P2 IN, P3 IN, P1 OUT, P3 IN

Sample Output

P1 SWAPPED IN. Memory available = 600
 P2 SWAPPED IN. Memory available = 300
 Not enough memory to swap in P3
 P1 SWAPPED OUT. Memory available = 700
 P3 SWAPPED IN. Memory available = 200

Question 8

Simulate segmentation memory management

Theory / Explanation

In segmentation, a program is divided into logically related units called segments (e.g., code, stack, data), each with its own base and limit stored in a segment table. A logical address consists of a segment number and an offset. The offset is validated against the segment's limit; if valid, the physical address is computed as base + offset, otherwise an addressing error occurs.

Pseudocode

```
BEGIN
  READ number_of_segments
  FOR each segment
    READ base[i], limit[i]
  READ segment_number, offset
  IF offset < limit[segment_number] THEN
    physical = base[segment_number] + offset
    PRINT physical
  ELSE
    PRINT "Error: offset exceeds segment limit"
  END IF
END
```

C Program (Code)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of segments: ");
    scanf("%d", &n);

    int base[n], limit[n];
    for (int i = 0; i < n; i++) {
        printf("Enter base and limit for segment %d: ", i);
        scanf("%d %d", &base[i], &limit[i]);
    }

    int seg, offset;
    printf("Enter segment number and offset: ");
    scanf("%d %d", &seg, &offset);

    if (seg >= 0 && seg < n) {
        if (offset >= 0 && offset < limit[seg]) {
            printf("Valid reference. Physical Address = %d\n", base[seg] + offset);
        } else {
            printf("Error: Offset exceeds segment limit (Trap)\n");
        }
    } else {
        printf("Error: Invalid segment number\n");
    }
    return 0;
}
```

Sample Input

```
Segments = 4  
Seg0: base=1219, limit=750  
Seg1: base=2300, limit=400  
Seg2: base=90, limit=100  
Seg3: base=1327, limit=580  
Query: segment=1, offset=53
```

Sample Output

```
Valid reference. Physical Address = 2353
```

Question 9

Demonstrate segmentation with paging

Theory / Explanation

Segmentation with paging combines both techniques: each segment is itself divided into fixed-size pages. A logical address has three parts - segment number, page number, and offset. The segment number indexes a segment table to find that segment's page table; the page number indexes the page table to obtain a frame number; the physical address is then $\text{frame_number} * \text{page_size} + \text{offset}$. This reduces external fragmentation while preserving the logical structure of segmentation.

Pseudocode

```
BEGIN
  READ page_size
  READ number_of_segments
  FOR each segment
    READ number_of_pages_in_segment
    READ page_table[segment][] // frame numbers
  READ segment_number, page_number, offset
  IF segment_number valid AND page_number valid AND offset < page_size THEN
    frame = page_table[segment_number][page_number]
    physical = frame * page_size + offset
    PRINT physical
  ELSE
    PRINT "Invalid reference"
  END IF
END
```

C Program (Code)

```
#include <stdio.h>

#define MAXSEG 10
#define MAXPAGE 10

int main() {
  int page_size, segs;
  int pageTable[MAXSEG][MAXPAGE];
  int numPages[MAXSEG];

  printf("Enter page size: ");
  scanf("%d", &page_size);
  printf("Enter number of segments: ");
  scanf("%d", &segs);

  for (int s = 0; s < segs; s++) {
    printf("Enter number of pages in segment %d: ", s);
    scanf("%d", &numPages[s]);
    for (int p = 0; p < numPages[s]; p++) {
      printf("Enter frame number for segment %d page %d: ", s, p);
      scanf("%d", &pageTable[s][p]);
    }
  }

  int seg, page, offset;
  printf("Enter segment number, page number, offset: ");
  scanf("%d %d %d", &seg, &page, &offset);
```

```
if (seg >= 0 && seg < segs && page >= 0 && page < numPages[seg] && offset < page_size) {  
    int frame = pageTable[seg][page];  
    int physical = frame * page_size + offset;  
    printf("Valid reference. Physical Address = %d\n", physical);  
} else {  
    printf("Invalid reference - segmentation/paging fault\n");  
}  
return 0;  
}
```

Sample Input

```
Page size = 256  
Segments = 2  
Segment 0: pages = 3, frames = [4, 7, 2]  
Segment 1: pages = 2, frames = [9, 1]  
Query: segment=0, page=1, offset=50
```

Sample Output

```
Valid reference. Physical Address = 1842
```

Question 10

Simulate virtual memory using demand paging

Theory / Explanation

Demand paging loads a page into memory only when it is referenced (not in advance). If a referenced page is already present, it is a hit; otherwise a page fault occurs and the page must be loaded from secondary storage into a free frame (or a frame is chosen for replacement if memory is full). The program shows frame contents after every reference and counts page faults.

Pseudocode

```
BEGIN
  READ page_reference_string
  READ number_of_frames
  frames = empty list of size number_of_frames
  page_faults = 0
  FOR each page in reference string
    IF page NOT IN frames THEN
      page_faults += 1
      IF frames not full THEN
        add page to frames
      ELSE
        remove oldest page from frames (FIFO)
        add page to frames
      END IF
    END IF
    PRINT current frame contents
  END FOR
  PRINT total page_faults
END
```

C Program (Code)

```
#include <stdio.h>

int main() {
  int n, frames_count;
  printf("Enter number of page references: ");
  scanf("%d", &n);
  int pages[n];
  printf("Enter reference string: ");
  for (int i = 0; i < n; i++) scanf("%d", &pages[i]);

  printf("Enter number of frames: ");
  scanf("%d", &frames_count);
  int frames[frames_count];
  for (int i = 0; i < frames_count; i++) frames[i] = -1;

  int faults = 0, next = 0;

  for (int i = 0; i < n; i++) {
    int found = 0;
    for (int j = 0; j < frames_count; j++)
      if (frames[j] == pages[i]) { found = 1; break; }

    if (!found) {
      frames[next] = pages[i];
      next = (next + 1) % frames_count;
    }
  }
}
```

```

        faults++;
        printf("Page %d -> FAULT | Frames: ", pages[i]);
    } else {
        printf("Page %d -> HIT   | Frames: ", pages[i]);
    }

    for (int j = 0; j < frames_count; j++) {
        if (frames[j] == -1) printf("- ");
        else printf("%d ", frames[j]);
    }
    printf("\n");
}
printf("\nTotal Page Faults = %d\n", faults);
return 0;
}

```

Sample Input

Reference string = 1 2 3 4 1 2 5 1 2 3 4 5
 Frames = 3

Sample Output

```

Page 1 -> FAULT | Frames: 1 - -
Page 2 -> FAULT | Frames: 1 2 -
Page 3 -> FAULT | Frames: 1 2 3
Page 4 -> FAULT | Frames: 4 2 3
Page 1 -> FAULT | Frames: 4 1 3
Page 2 -> FAULT | Frames: 4 1 2
Page 5 -> FAULT | Frames: 5 1 2
Page 1 -> HIT   | Frames: 5 1 2
Page 2 -> HIT   | Frames: 5 1 2
Page 3 -> FAULT | Frames: 5 3 2
Page 4 -> FAULT | Frames: 5 3 4
Page 5 -> HIT   | Frames: 5 3 4

```

Total Page Faults = 9

Question 11

Compare FIFO, LRU and Optimal page replacement algorithms

Theory / Explanation

FIFO replaces the page that has been in memory the longest, regardless of usage. LRU (Least Recently Used) replaces the page that has not been used for the longest time, approximating future behaviour from the past. Optimal replaces the page that will not be used for the longest time in the future (theoretical best, used as a benchmark). All three are simulated on the same reference string and frame count, and their page-fault counts are compared.

Pseudocode

```
BEGIN
  READ reference_string, number_of_frames
  // FIFO
  use a queue; on fault, remove the page at front of queue and insert new page
  // LRU
  on fault, remove the page whose last-used time is smallest (farthest in the past)
  // Optimal
  on fault, remove the page whose next use is farthest in the future (or never used again)
  PRINT frame contents after each reference and total faults for each algorithm
END
```

C Program (Code)

```
#include <stdio.h>

int isPresent(int frames[], int fc, int page) {
    for (int i = 0; i < fc; i++) if (frames[i] == page) return 1;
    return 0;
}

void fifo(int pages[], int n, int fc) {
    int frames[fc], faults = 0, next = 0;
    for (int i = 0; i < fc; i++) frames[i] = -1;
    for (int i = 0; i < n; i++) {
        if (!isPresent(frames, fc, pages[i])) {
            frames[next] = pages[i];
            next = (next + 1) % fc;
            faults++;
        }
    }
    printf("FIFO Page Faults = %d\n", faults);
}

void lru(int pages[], int n, int fc) {
    int frames[fc], lastUsed[fc], faults = 0;
    for (int i = 0; i < fc; i++) frames[i] = -1;
    for (int i = 0; i < n; i++) {
        if (!isPresent(frames, fc, pages[i])) {
            int idx = -1, minTime = 1e9;
            for (int j = 0; j < fc; j++) {
                if (frames[j] == -1) { idx = j; break; }
                if (lastUsed[j] < minTime) { minTime = lastUsed[j]; idx = j; }
            }
            frames[idx] = pages[i];
            faults++;
        }
    }
}
```

```

        for (int j = 0; j < fc; j++)
            if (frames[j] == pages[i]) lastUsed[j] = i;
    }
    printf("LRU Page Faults = %d\n", faults);
}

void optimal(int pages[], int n, int fc) {
    int frames[fc], faults = 0;
    for (int i = 0; i < fc; i++) frames[i] = -1;
    for (int i = 0; i < n; i++) {
        if (!isPresent(frames, fc, pages[i])) {
            int idx = -1, farthest = -1;
            for (int j = 0; j < fc; j++) {
                if (frames[j] == -1) { idx = j; break; }
                int k;
                for (k = i + 1; k < n; k++)
                    if (frames[j] == pages[k]) break;
                if (k > farthest) { farthest = k; idx = j; }
            }
            frames[idx] = pages[i];
            faults++;
        }
    }
    printf("Optimal Page Faults = %d\n", faults);
}

int main() {
    int n, fc;
    printf("Enter number of page references: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &fc);

    fifo(pages, n, fc);
    lru(pages, n, fc);
    optimal(pages, n, fc);
    return 0;
}

```

Sample Input

```

Reference string = 7 0 1 2 0 3 0 4 2 3 0 3 2
Frames = 3

```

Sample Output

```

FIFO Page Faults = 9
LRU Page Faults = 8
Optimal Page Faults = 7

```


Question 12

Simulate thrashing and working-set management

Theory / Explanation

Thrashing occurs when a process spends more time paging (swapping pages in and out) than executing, usually because it does not have enough frames to hold its working set. The working set is the set of pages a process actively references within a moving time window (Delta). By calculating the working-set size at different points in the reference string, the program can identify moments when the working set exceeds available frames, signalling potential thrashing.

Pseudocode

```
BEGIN
  READ reference_string, window_size (Delta), available_frames
  FOR i = window_size TO length(reference_string)
    window = reference_string[i-window_size ... i-1]
    working_set = DISTINCT pages in window
    PRINT working_set, size(working_set)
    IF size(working_set) > available_frames THEN
      PRINT "Thrashing risk at this interval"
    END IF
  END FOR
END
```

C Program (Code)

```
#include <stdio.h>

int main() {
    int n, delta, frames;
    printf("Enter length of reference string: ");
    scanf("%d", &n);
    int ref[n];
    printf("Enter reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &ref[i]);

    printf("Enter working-set window size (Delta): ");
    scanf("%d", &delta);
    printf("Enter number of available frames: ");
    scanf("%d", &frames);

    for (int i = delta; i <= n; i++) {
        int ws[50], wsCount = 0;
        for (int j = i - delta; j < i; j++) {
            int found = 0;
            for (int k = 0; k < wsCount; k++)
                if (ws[k] == ref[j]) { found = 1; break; }
            if (!found) ws[wsCount++] = ref[j];
        }
        printf("Window ending at t=%d -> Working Set = {", i);
        for (int k = 0; k < wsCount; k++) printf("%d ", ws[k]);
        printf("}, WS Size = %d", wsCount);
        if (wsCount > frames) printf(" <-- THRASHING RISK");
        printf("\n");
    }
    return 0;
}
```

Sample Input

```
Reference string = 1 2 3 4 2 1 5 6 2 1  
Delta = 4  
Available frames = 3
```

Sample Output

```
Window ending at t=4 -> Working Set = {1 2 3 4 }, WS Size = 4 <-- THRASHING RISK  
Window ending at t=5 -> Working Set = {2 3 4 }, WS Size = 3  
Window ending at t=6 -> Working Set = {3 4 2 1 }, WS Size = 4 <-- THRASHING RISK  
Window ending at t=7 -> Working Set = {4 2 1 5 }, WS Size = 4 <-- THRASHING RISK  
Window ending at t=8 -> Working Set = {2 1 5 6 }, WS Size = 4 <-- THRASHING RISK  
Window ending at t=9 -> Working Set = {1 5 6 2 }, WS Size = 4 <-- THRASHING RISK  
Window ending at t=10 -> Working Set = {5 6 2 1 }, WS Size = 4 <-- THRASHING RISK
```

Question 13

Simulate dynamic linking using a symbol table of library functions

Theory / Explanation

In dynamic linking, references to library functions are resolved at run time rather than at compile time. The operating system/loader maintains a symbol table listing each library function along with the library it belongs to and its memory address. When a running program calls a function, the loader looks it up in the symbol table; if found, the function is linked (loaded once and shared) and its address is returned, otherwise an 'unresolved symbol' error is reported.

Pseudocode

```
BEGIN
  CREATE symbol_table with entries (function_name, library_name, memory_address)
  READ requested_function
  IF requested_function EXISTS in symbol_table THEN
    PRINT library_name, memory_address
    PRINT "Function dynamically linked"
  ELSE
    PRINT "Error: Unresolved external symbol"
  END IF
END
```

C Program (Code)

```
#include <stdio.h>
#include <string.h>

struct Symbol {
    char func[30];
    char lib[30];
    int address;
};

int main() {
    struct Symbol table[] = {
        {"printf", "libc.so", 4001},
        {"malloc", "libc.so", 4050},
        {"sqrt", "libm.so", 5010},
        {"pthread_create", "libpthread.so", 6020}
    };
    int n = sizeof(table) / sizeof(table[0]);

    char query[30];
    printf("Symbol Table:\n");
    for (int i = 0; i < n; i++)
        printf(" %-16s -> %-16s @ %d\n", table[i].func, table[i].lib, table[i].address);

    printf("\nEnter function name requested at run time: ");
    scanf("%s", query);

    int found = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(table[i].func, query) == 0) {
            printf("Symbol '%s' resolved.\n", query);
            printf("Library = %s, Memory Address = %d\n", table[i].lib, table[i].address);
            printf("Function dynamically linked at run time.\n");
        }
    }
}
```

```
        found = 1;
        break;
    }
}
if (!found)
    printf("Error: Unresolved external symbol '%s'\n", query);

return 0;
}
```

Sample Input

Requested function at run time: sqrt

Sample Output

Symbol Table:

printf	-> libc.so	@ 4001
malloc	-> libc.so	@ 4050
sqrt	-> libm.so	@ 5010
pthread_create	-> libpthread.so	@ 6020

Symbol 'sqrt' resolved.
Library = libm.so, Memory Address = 5010
Function dynamically linked at run time.

Question 14

Simulate a linker using a simplified two-pass approach

Theory / Explanation

A two-pass linker resolves symbols across multiple object modules. In Pass 1, it scans every module, records each module's starting address (based on cumulative length of previous modules), and builds an External Symbol Table (EST) mapping each exported symbol to its absolute address. In Pass 2, it scans each module's reference list and replaces every external reference with the resolved absolute address from the EST, producing the final linked addresses; unresolved symbols are reported as errors.

Pseudocode

```
BEGIN
  // Pass 1
  address = 0
  FOR each object module M
    base_address[M] = address
    FOR each defined symbol S in M
      EST[S] = base_address[M] + relative_offset(S)
    address = address + length(M)
  END FOR

  // Pass 2
  FOR each object module M
    FOR each external reference R in M
      IF R exists in EST THEN
        resolved_address = EST[R]
        PRINT R, "resolved to", resolved_address
      ELSE
        PRINT "Error: unresolved symbol", R
      END IF
    END FOR
  END FOR
END
```

C Program (Code)

```
#include <stdio.h>
#include <string.h>

struct Symbol { char name[20]; int address; };

int main() {
  int m;
  printf("Enter number of object modules: ");
  scanf("%d", &m);

  int length[m], base[m];
  struct Symbol EST[50];
  int estCount = 0;

  for (int i = 0; i < m; i++) {
    printf("Enter length of module %d: ", i + 1);
    scanf("%d", &length[i]);
  }

  // Pass 1: compute base addresses and build EST
  base[0] = 0;
```

```

    for (int i = 1; i < m; i++) base[i] = base[i - 1] + length[i - 1];

    for (int i = 0; i < m; i++) {
        int defCount;
        printf("Module %d - Enter number of defined symbols: ", i + 1);
        scanf("%d", &defCount);
        for (int j = 0; j < defCount; j++) {
            char name[20]; int offset;
            printf("    Enter symbol name and relative offset: ");
            scanf("%s %d", name, &offset);
            strcpy(EST[estCount].name, name);
            EST[estCount].address = base[i] + offset;
            estCount++;
        }
    }

    printf("\n-- External Symbol Table (Pass 1) --\n");
    for (int i = 0; i < estCount; i++)
        printf("%s -> %d\n", EST[i].name, EST[i].address);

    // Pass 2: resolve references
    printf("\n-- Resolving References (Pass 2) --\n");
    for (int i = 0; i < m; i++) {
        int refCount;
        printf("Module %d - Enter number of external references: ", i + 1);
        scanf("%d", &refCount);
        for (int j = 0; j < refCount; j++) {
            char ref[20];
            printf("    Enter referenced symbol: ");
            scanf("%s", ref);
            int found = 0;
            for (int k = 0; k < estCount; k++) {
                if (strcmp(EST[k].name, ref) == 0) {
                    printf("    %s resolved to address %d\n", ref, EST[k].address);
                    found = 1;
                    break;
                }
            }
            if (!found) printf("    Error: unresolved symbol %s\n", ref);
        }
    }
    return 0;
}

```

Sample Input

```

Modules = 2
Module1 length=100, defines: A(off=10)
Module2 length=80,  defines: B(off=20)
Module1 references: B
Module2 references: A, C

```

Sample Output

```

-- External Symbol Table (Pass 1) --
A -> 10
B -> 120

-- Resolving References (Pass 2) --
B resolved to address 120

```

A resolved to address 10
Error: unresolved symbol C

Question 15

Simulate disk scheduling algorithms - FCFS, SSTF, SCAN and C-SCAN

Theory / Explanation

Disk scheduling algorithms decide the order in which pending disk I/O requests are serviced to minimize head movement. FCFS services requests strictly in arrival order. SSTF (Shortest Seek Time First) always services the request closest to the current head position. SCAN moves the head in one direction servicing requests, reverses at the disk boundary, and continues, like an elevator. C-SCAN also sweeps in one direction, but on reaching the end it jumps back to the beginning without servicing requests on the return trip, giving more uniform wait times.

Pseudocode

```
BEGIN
  READ requests[], head_position, disk_size
  // FCFS
  service requests strictly in the given order; sum absolute differences
  // SSTF
  repeatedly pick the unserviced request nearest to current head; sum differences
  // SCAN
  sort requests; move head in one direction to the end/boundary, then reverse and
  service remaining requests on the way back; sum total movement
  // C-SCAN
  sort requests; move head to the end in one direction servicing requests,
  jump to the beginning, then continue upward servicing remaining requests
  PRINT service order and total head movement for each algorithm
END
```

C Program (Code)

```
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) { return (*(int*)a - *(int*)b); }

void fcfs(int req[], int n, int head) {
    int total = 0, cur = head;
    printf("\nFCFS order: %d", head);
    for (int i = 0; i < n; i++) {
        total += abs(req[i] - cur);
        cur = req[i];
        printf(" -> %d", cur);
    }
    printf("\nTotal head movement (FCFS) = %d\n", total);
}

void sstf(int req[], int n, int head) {
    int visited[n], total = 0, cur = head;
    for (int i = 0; i < n; i++) visited[i] = 0;
    printf("\nSSTF order: %d", head);
    for (int c = 0; c < n; c++) {
        int idx = -1, minDist = 1e9;
        for (int i = 0; i < n; i++)
            if (!visited[i] && abs(req[i] - cur) < minDist) {
                minDist = abs(req[i] - cur); idx = i;
            }
        total += minDist;
        cur = req[idx];
        visited[idx] = 1;
    }
}
```



```

        printf(" -> %d", cur);
    }
    printf("\nTotal head movement (SSTF) = %d\n", total);
}

void scan(int req[], int n, int head, int diskSize) {
    int r[n + 1];
    for (int i = 0; i < n; i++) r[i] = req[i];
    r[n] = head;
    qsort(r, n + 1, sizeof(int), cmp);

    int idx = 0;
    for (int i = 0; i <= n; i++) if (r[i] == head) { idx = i; break; }

    int total = 0, cur = head;
    printf("\nSCAN order: %d", head);
    for (int i = idx + 1; i <= n; i++) { total += abs(r[i]-cur); cur = r[i]; printf(" -> %d",
cur); }
    total += abs((diskSize - 1) - cur); cur = diskSize - 1; printf(" -> %d(end)", cur);
    for (int i = idx - 1; i >= 0; i--) { total += abs(r[i]-cur); cur = r[i]; printf(" -> %d",
cur); }
    printf("\nTotal head movement (SCAN) = %d\n", total);
}

void cscan(int req[], int n, int head, int diskSize) {
    int r[n];
    for (int i = 0; i < n; i++) r[i] = req[i];
    qsort(r, n, sizeof(int), cmp);

    int total = 0, cur = head;
    printf("\nC-SCAN order: %d", head);
    int i = 0;
    while (i < n && r[i] < head) i++;
    for (int j = i; j < n; j++) { total += abs(r[j]-cur); cur = r[j]; printf(" -> %d", cur); }
    total += abs((diskSize - 1) - cur); cur = diskSize - 1; printf(" -> %d(end)", cur);
    total += (diskSize - 1); cur = 0; printf(" -> 0(wrap)");
    for (int j = 0; j < i; j++) { total += abs(r[j]-cur); cur = r[j]; printf(" -> %d", cur); }
    printf("\nTotal head movement (C-SCAN) = %d\n", total);
}

int main() {
    int n, head, diskSize;
    printf("Enter number of requests: ");
    scanf("%d", &n);
    int req[n];
    printf("Enter disk requests: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size (max cylinder + 1): ");
    scanf("%d", &diskSize);

    fcfs(req, n, head);
    sstf(req, n, head);
    scan(req, n, head, diskSize);
    cscan(req, n, head, diskSize);
    return 0;
}

```

Sample Input

Requests = 98 183 37 122 14 124 65 67
Initial head = 53
Disk size = 200

Sample Output

FCFS order: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total head movement (FCFS) = 640

SSTF order: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movement (SSTF) = 236

SCAN order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199(end) -> 37 -> 14
Total head movement (SCAN) = 331

C-SCAN order: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199(end) -> 0(wrap) -> 14 -> 37
Total head movement (C-SCAN) = 382